**ASSIGNMENT 1**

**Course: B.Tech. CSE** **Semester:5**

**Subject: System Design** **Subject Code: TCS-504**

**Submission Date: 07-Septemeber-2026**

# 1. The Problem

Build a small movie ticket booking system for a single cinema (like PVR or INOX), as a menu-driven C++ console program.

A customer should be able to see which movies are playing, pick a show, see which seats are free, book seats, pay, get a ticket, and cancel a booking.

# 2. Scope

| Features | Description |
|---|---|
| F1 | List all movies currently playing. |
| F2 | For a chosen movie, list its shows (screen + start time). |
| F3 | For a chosen show, display the seat layout with available / booked status. |
| F4 | Book one or more seats for a show (reject an already-booked seat). |
| F5 | Price the booking by seat type: silver ₹150, gold ₹250, platinum ₹400. |
| F6 | Pay by UPI, card or cash — a failed payment must not confirm the booking. |
| F7 | Print a ticket: booking id, movie, screen, time, seat numbers, total amount. |
| F7 | Cancel a booking — the seats become available again. |

# 3. Requirement analysis

## a) Functional Requirements (FR)

| Functional Requirements | Description |
|---|---|
| FR 1 | **View Movie List:**<br>When the user selects "Movies" from the main menu, the program prints every movie currently in the theater along with its language and duration (e.g., "Name \| Language \| Duration").<br>If no movies are loaded, it shows a message saying no movies are playing right now. |
| FR 2 | **View Shows for a Movie:**<br>After picking a movie number, the user gets a list of all screens showing that movie along with the showtimes (e.g., "Screen-1 at 06:00 PM").<br>If the user enters an invalid show or movie number, the app shows an error message and asks them to choose again instead of crashing. |
| FR 3 | **View Seat Layout:**<br>Once a show is selected, the console displays a grid of seats organized by section (SILVER, GOLD, PLATINUM).<br>Available seats are shown as [ ] and already booked seats are marked as [X] so the user clearly sees what is open. |
| FR 4 | **Seat Selection & Lock:**<br>A customer selects one or more seat numbers for a show. If any selected seat is already BOOKED, the whole booking is rejected and no seat changes state.<br>Booking is confirmed only after payment succeeds. |
| FR 5 | **Calculate Total Price:**<br>The system checks the seat category for every seat chosen (SILVER = Rs 150, GOLD = Rs 250, PLATINUM = Rs 400) and prints an itemized breakdown with the final total amount before asking for payment. |
| FR 6 | **Payment Handling:**<br>Exactly one method (UPI/Card/Cash) per booking.<br>If payment fails, seats are released and booking status becomes FAILED. |
| FR 7 | **Print Ticket:**<br>As soon as payment succeeds, the system displays a formatted ticket on the screen with a generated Booking ID, movie name, screen, showtime, seat numbers, total price paid, and status set to CONFIRMED. |
| FR 8 | **Cancel Booking:**<br>The user can enter their Booking ID to cancel a ticket. The system updates the booking status to CANCELLED and immediately frees up those exact seats ([X] turns back to [ ]) for future bookings. |

## b) Non-Functional Requirements (NFR)

| Non-Functional Requirements | Description |
|---|---|
| NFR 1 | **One Class Per File:**<br>Keep code organized by putting each class in its own dedicated source file (e.g., 01_Movie.cpp, 02_Seat.cpp). |
| NFR 2 | **Easy to Extend (Open/Closed):**<br>Use an abstract Payment base class so adding a new payment method (like NetBanking or Wallet) later only requires creating a new derived class without modifying existing Booking code. |
| NFR 3 | **Safe Console Input:**<br>Handle invalid inputs safely (like entering letters for numeric menus or non-existent seat codes).<br>The program should clear input buffers (cin.clear(), cin.ignore()) and re-prompt the user instead of crashing or looping indefinitely. |
| NFR 4 | **Transactional State Consistency:**<br>If a booking fails or a payment is declined midway, all temporarily selected seats must immediately revert to AVAILABLE [ ] so no seat stays locked incorrectly. |
| NFR 5 | **Unique ID Generation:**<br>Use a shared static counter inside the Booking class to auto-increment unique IDs (e.g., BK1001, BK1002) so no two tickets ever share the same identifier. |

# 4. Noun-Verb Analysis

## a) Noun Analysis (Classes & Attributes)

| Noun Found | Keep as Class? | Reason |
|---|---|---|
| Movie | Yes | Has its own identity, title, language, and duration. |
| Seat | Yes | Represents a physical chair with a seat number and category (SILVER, GOLD, PLATINUM). |
| "seat layout" | No | It is a printed visual view of a Show's seat grid, handled by a method inside Show or BookingService. |
| Cinema | Yes | Top-level entity that owns and manages auditorium screens. |
| Screen | Yes | Represents an auditorium hall that owns a physical set of seats. |
| Show | Yes | Binds a Movie to a Screen at a specific time slot. |

| | | |
|---|---|---|
| ShowSeat | Yes | Tracks the dynamic state (AVAILABLE or BOOKED) of a physical seat for a specific show. |
| Customer | Yes | Holds user details such as name and contact phone number. |
| Booking | Yes | Stores a confirmed or failed transaction record (Booking ID, show, seats, amount, status). |
| Payment | Yes | Abstract base entity that defines the payment interface (pay(amount)). |
| UPI / Card / Cash | Yes | Derived concrete classes implementing specific payment methods. |
| Price Calculator | Yes | Service class dedicated solely to calculating ticket prices based on seat tiers. |
| Ticket Printer | Yes | Service class dedicated strictly to formatting and displaying ticket receipts. |
| Booking Service | Yes | Core orchestrator managing the business workflow (booking, payment validation, cancellation). |
| "booking id" | No | Primitive attribute (string) inside the Booking class, not an independent entity. |
| "amount/ price" | No | Primitive numerical property (double), managed by PriceCalculator and Booking. |
| "menu/ choice" | No | Console UI interaction mechanism, handled inside main.cpp. |

## b) Verb Analysis (Methods & Operations)

| Verb Found | Associated Class | Method Name | Responsibility |
|---|---|---|---|
| "list movies" | BookingService | getMovies() / displayMovies() | Fetches active movies from cinema inventory. |
| "list shows" | BookingService | getShowsForMovie() | Filters and returns shows matching a movie. |
| "display layout" | Show | displaySeatLayout() | Prints the matrix of seats showing AVAILABLE vs BOOKED. |
| "book seats" | ShowSeat / BookingService | lockSeat() / createBooking() | Marks selected seats as BOOKED if currently free. |
| "calculate price" | PriceCalculator | calculateTotal() | Sums prices of chosen seat types (SILVER, GOLD, PLATINUM). |

| | | | |
|---|---|---|---|
| "pay" | Payment | pay(double amount) | Executes transaction logic for UPI, Card, or Cash. |
| "print ticket" | TicketPrinter | printTicket() | Formats and outputs confirmed booking receipt. |
| "cancel booking" | BookingService / Booking | cancelBooking() / cancel() | Changes booking status to CANCELLED and frees seats back to AVAILABLE. |

# 5. Classes Required

These are the classes required.

## a. Core Entity Classes

| Class | It's One Responsibility |
|---|---|
| Movie | Title, language, and duration — nothing else. |
| Seat | One physical seat: seat number and category type (SILVER/GOLD/PLATINUM) — nothing else. |
| Screen | Screen identifier and its collection of physical seats — nothing else. |
| Cinema | Theater name and its list of screens — nothing else. |
| Show | One specific time slot mapping a Movie to a Screen with its seat availability layout — nothing else. |
| ShowSeat | The runtime availability status (AVAILABLE vs BOOKED) of one physical seat for one specific show — nothing else. |
| Customer | User identification details (customer name and phone number) — nothing else. |
| Booking | Transaction record holding Booking ID, Customer, Show, booked seats, total amount, and booking status — nothing else. |

## b. Behavior and Service Classes

| Class | It's One Responsibility |
|---|---|
| Payment | Abstract base contract declaring the pure virtual pay() method for payment methods — nothing else. |
| PaymentTypes (UpiPayment / CardPayment / CashPayment) | Concrete execution logic for processing payments via UPI, Card, or Cash — nothing else. |

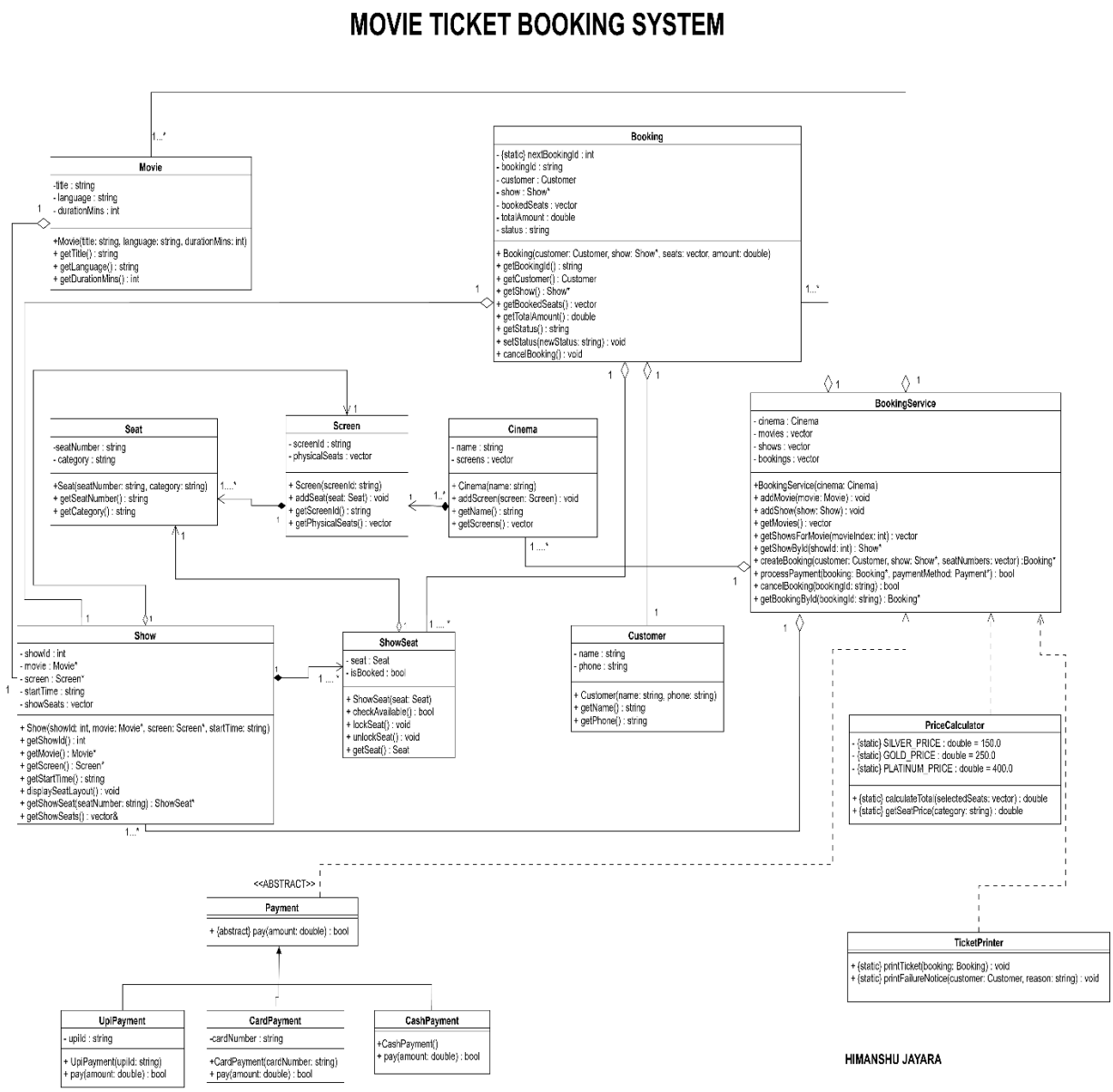
| | |
|---|---|
| PriceCalculator | Pure calculation logic that computes the total booking cost based on seat categories — nothing else. |
| TicketPrinter | Formatting and displaying the ticket receipt on the console — nothing else. |
| BookingService | Orchestrating the core business workflow for seat reservation, payment execution, and cancellation — nothing else. |

## 7. Relationships

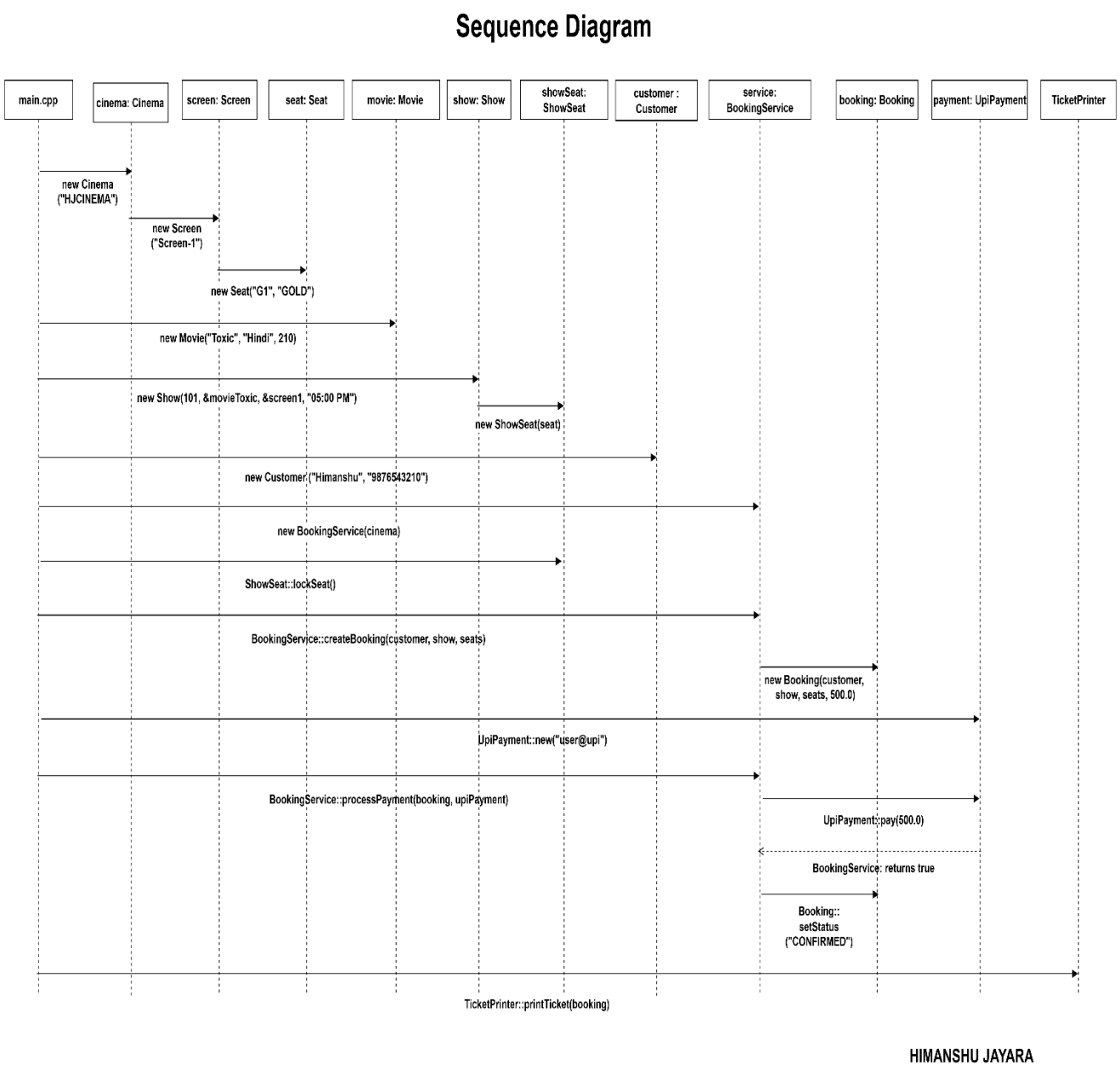
| PAIR | RELATION | JUSTIFICATION |
|---|---|---|
| Cinema → Screen | Composition | A Screen exists inside a specific Cinema. If the cinema building is demolished, all its screens are destroyed with it. |
| Screen → Seat | Composition | Physical seats belong exclusively to a specific screen. Destroying the screen auditorium destroys its physical seats. |
| Show → Movie | Aggregation | A Show borrows a Movie. Cancelling or removing a show time slot does NOT delete the Movie from the theater's library. |
| Show → Screen | Aggregation | A Show references a Screen for a time slot. Cancelling the 6 PM show does NOT destroy the physical screen hall. |
| Show → ShowSeat | Composition | ShowSeat tracks seat availability for *one specific show*. If the show is deleted, its unique seat-status matrix dies with it. |
| Booking → Customer | **Aggregation** | A Booking holds a reference to a Customer. Cancelling or deleting a booking record does NOT delete the customer from the system. |
| Booking → ShowSeat | **Aggregation** | A Booking references ShowSeat objects. Cancelling a booking updates seat status to AVAILABLE; it does NOT destroy the physical seats. |
| Booking → Payment | Association | A Booking uses a Payment object to execute the transaction. Payment is a transient service, not a component owned long-term by Booking. |

| | | |
|---|---|---|
| Payment → UpiPayment, CardPayment, CashPayment) | Inheritance | UpiPayment *is-a* concrete implementation of the abstract Payment class (UpiPayment , CardPayment, CashPayment extends Payment). |
| BookingService → Booking | **Aggregation** | BookingService manages a list of Booking records. The bookings exist as business domain records independently of the service engine. |

# 8. Class diagram



# 9. Sequence diagram




HIMANSHU JAYARA

# 10. Modular code

```
CinemaBookingSystem/
├── Seat.cpp
├── Screen.cpp
├── Movie.cpp
├── ShowSeat.cpp
├── Show.cpp
├── Customer.cpp
├── Payment.cpp
├── UpiPayment.cpp
├── CardPayment.cpp
├── CashPayment.cpp
├── Booking.cpp
├── BookingService.cpp
├── TicketPrinter.cpp
└── main.cpp
```

## A. Verification Checklist against Requirements

| **Requirement** | Implementation Details | Status |
|---|---|---|
| One class per file | Every class (Seat, Screen, Movie, etc.) lives in its own standalone .cpp file. | In progress. |
| No header files | .cpp files include implementation natively; #include "*.cpp" in main.cpp allows compilation without .h headers. | In progress. |
| Encapsulation | seatStatus, bookingAmount, status are private and altered via dedicated guard methods (lockSeat, confirmSeat, setStatus). | In progress. |
| Abstraction | Pure virtual function virtual bool pay(double amount) = 0; inside abstract Payment class. | In progress. |
| Inheritance | UpiPayment, CardPayment, CashPayment inherit publicly from Payment. | In progress. |
| Runtime Polymorphism | Handled inside BookingService::processPayment(..., Payment* paymentMethod) via paymentMethod->pay(). | In progress. |
| Compile-Time Polymorphism | Overloaded constructors in Movie class. | In progress. |

| | | |
|---|---|---|
| Static Members | static int nextBookingId; in Booking class auto-increments for each new transaction. | In progress. |
| this Keyword | Used inside constructors across classes (this->seatNumber, this->title, this->name). | In progress. |
| Composition | Cinema → Screen, Screen → Seat, Show → ShowSeat. | In progress. |
| Aggregation | Show → Movie, Booking → ShowSeat. | In progress. |
| Association | Customer → BookingService. | In progress. |
| Edge Case 1 | Duplicate booking attempt on seat G1 is rejected. | In progress. |
| Edge Case 2 | Payment with "invalid@upi" triggers status revert and seat unlocking. | In progress. |
| Edge Case 3 | cancelBooking() marks booking cancelled and makes seats AVAILABLE again. | In progress. |
| Edge Case 4 | Requesting non-existent seat Z99 outputs a clear error without crashing. | In progress. |

# 11.SOLID

| PRINCIPLE | CONCEPT | USE IN CODE |
|---|---|---|
| **Single Responsibility Principle (SRP)** | A class should have **one, and only one, reason to change**. | BookingService should only focus on managing booking logic and pricing calculations.<br>Printing layout, formatting receipts, or user interface presentation should be handled by a completely separate class (TicketPrinter).<br>If ticket layout requirements change, you only edit TicketPrinter, leaving BookingService untouched. |
| Open-Closed Principle (OCP) | Software entities should be **open for extension**, but **closed for modification**. | I can add new functionality (such as a new payment method like NetBanking) by writing a new class, without altering or risking breaking existing classes like BookingService or CardPayment. |
| Liskov Substitution Principle (LSP) | Subtypes must be **substitutable for their base types** without breaking the application or requiring special-case checks. | Every payment subtype (CardPayment, CashPayment, NetBanking) must execute through a base pointer/reference (Payment* or Payment&) effortlessly, without requiring extra initialization methods, flags, or type checks before invoking processPayment(). |
| Interface Segregation Principle (ISP) | Clients should **not be forced to depend upon interfaces they do not use**. | Not all payment methods support online refunds (e.g., cash payments at a counter). Instead of forcing a dummy refund() implementation onto every payment class via the main Payment interface, split Refundable into a separate interface that only online payment methods implement. |
| Dependency Inversion Principle (DIP) | High-level modules should **not depend on low-level modules**. Both should depend on **abstractions**. | BookingService (high-level) should not directly instantiate low-level payment classes using new CardPayment(). Instead, BookingService accepts an abstract Payment& dependency via method or constructor parameter injection. |

## 12. Clean Code Checklist

- [x] **Intention-revealing names** — Use descriptive names like bookedSeatCount, selectedSeatIndex, and totalPaymentAmount instead of short or ambiguous variables like bsc, c1, or temp.
- [x] **No number-series names** — Eliminates names like list1, list2, temp1, or str2. Collections and variables must describe their entity (e.g., seatList, availableSeats).
- [x] **Every function does ONE thing** — Functions must not mix concerns. For example, calculateTotalPrice() only computes prices, while processPayment() only handles transaction logic.
- [x] **No function longer than ~20 lines** — Helper logic is extracted into smaller, focused routines to keep all methods short and easy to read.
- [x] **Maximum 2 levels of indentation** — Use early returns (guard clauses) and descriptive boolean expressions instead of deep nested if/else logic.
- [x] **0–2 parameters preferred** — Methods take at most 2 parameters. If a function requires more parameters, bundle them into a dedicated struct or class (e.g., BookingRequest).
- [x] **No side effects** — Read-only functions (like showSeats() or calculateTotal()) must not modify application state, book seats, or trigger transactions.
- [x] **Constants instead of magic numbers** — Replace raw literals with named constants:
- [x] **No repeated code (DRY - Don't Repeat Yourself)** — Input parsing, validation, and layout formatting are centralized into single reusable functions rather than duplicated across classes.